

Analiza wyników pomiarowych z prób spęczania z wykorzystaniem aplikacji komputerowej

Szymon Miłkowski

12.01.2026

1 Cel i wstępne założenia

Celem tego projektu było stworzenie graficznego programu komputerowego do analizy, korekcji i transformacji danych pomiarowych z prób spęczania. Danymi wejściowymi do programu miały być pliki binarne w formacie *W01* generowane przez oprogramowanie maszyny badawczej.

Dodatkowym celem miała być kompatybilność i płynność działania zarówno na współczesnym sprzęcie, jak i starszym (Windows XP 32-bit).

2 Wybór technologii

Z racji na wymóg szerokiej kompatybilności, oraz wysokiej wydajności, konieczne było wybranie technologii pozwalającej na kompilację do natywnego kodu maszynowego z minimalną ilością zależności. Wybór padł na język C z biblioteką Raylib do obsługi grafiki, linkowaną statycznie.

Raylib posiada łatwe w obsłudze API składające się z funkcji do rysowania i obsługi wejścia (mysz, klawiatura).

3 Spęczanie

Próba spęczania polega na ściskaniu próbki wzdłuż jednej osi (pionowej) przez maszynę badawczą z zadaną prędkością. Podczas próby materiał odkształca się, zmniejszając swoją wysokość w kierunku siły oraz - zgodnie z zasadą zachowania objętości - zwiększając swoją szerokość w pozostałych kierunkach.

Podczas tego procesu, materiał wewnątrz płynie, wypychając materiał na ścianach bocznych na zewnątrz, a materiał przy podstawach trze o górną i dolną ścianę maszyny, co hamuje jego przemieszczenie na zewnątrz.

Siła nacisku (opór jaki stawia próbka) jest mierzona i zależy od materiału i fazy procesu.

3.1 Fazy procesu

Brak kontaktu

- Brak kontaktu - maszyna nie ściska próbki, zmierza w jej kierunku - stała, niska siła
- Pierwszy kontakt - luzy w maszynie są ściskane - szum informacyjny
- Faza liniowa?? - siła rośnie liniowo
- Faza nieliniowa?? - siła rośnie coraz mniej

4 Format pliku wejściowego

4.1 Nieznany format binarny

Pliki wejściowe z danymi pomiarowymi z rozszerzeniem *.W01*, zapisane są w nieudokumentowanym formacie binarnym. Wpisywanie w wyszukiwarce *.W01* lub magic number z pliku - pierwsze cztery bajty *66 66 86 3f* nie daje istotnych rezultatów. Aby zczytać dane z plików i stworzyć algorytm importujący do programu, należy więc przeprowadzić inżynierię wsteczną formatu.

4.2 Inżynieria wsteczna formatu

W inżynierii wstecznej nie ma jednego poprawnego przepisu na znalezienie rozwiązania. Każde zadanie jest osobną łamigłówką, do której musimy użyć narzędzi, doświadczenia, wiedzy informatycznej, wiedzy dziedzinowej oraz czasu. Podczas analizy od dyspozycji miałem dwa pliki: *19.v10.W01* oraz *20.v50.W01*, gdzie *v10* i *v50* odpowiadają prędkości procesu spęczania.

4.2.1 Pierwsze spojrzenie na plik

Rzut okiem w *hex-edytorze* ujawnia kilka stringów i sporo zer na początku pliku, a w dalszej części powtarzający się czterokrotnie wzór: napis i blok nieczytelnych danych.

4.2.2 Nagłówek

W nagłówku pliku, który znajduje się na początku, przed właściwymi danymi liczbowymi znajdują się metadane, informacje o rozmiarze, godzina utworzenia oraz nazwa programu wejściowego do maszyny badawczej. Cały nagłówek ma stały rozmiar 208 bajtów.

Struktura nagłówku pliku				
offset	rozmiar	wartość/typ	nazwa	komentarz
0	4	0x3f866666	magic	stała wartość określająca format
4	4	1	wersja?	najpewniej wersja formatu
8	4	$rozmiar_pliku + 1$	rozmiar	
12	4	0	nieznana	zawsze 0
16	8	"ScriptPl"	stały napis	8 znaków, bez null terminatora
24	4	$rozmiar_pliku - 16$	rozmiar2	wygląda jak zbędna informacja, może oznaczać "rozmiar począwszy od napisu"
28	4	liczba wierszy	liczba wierszy	ilość liczb w każdej kolumnie danych
32	4	liczba kolumn	liczba kolumn	ilość kolumn z danymi
36	4	1	nieznane	
40	4	1	nieznane	
44	18	"dd-MM-yyyyhh:mm:ss"	czas utworzenia	napis bez null terminatora
62	42	nieznana	nieznane	42 bajty, głównie zera
104	104	nazwa pliku	plik badania	nazwa pliku wejściowego do maszyny, dopełniona zerami do 104 bajtów

4.2.3 Kolumny (serie danych)

Kolumny mają zmienny rozmiar, zależnie od pliku, jednak zawsze taki sam w obrębie jednego pliku.

Struktura kolumny				
offset	rozmiar	wartość/typ	nazwa	komentarz
0	4	int	nieznane	1 lub 16385
4	59	napis	nazwa	nazwa i jednostka kolumny, oddzielone spacjami
63	$4 * liczba_wierszy$	float	dane	tablica 32-bitowych liczb zmiennoprzecinkowych

4.2.4 Struktura całego pliku

Plik składa się z nagłówka o stałym rozmiarze i zmiennej liczby kolumn zmiennej długości, które zawierają swój nagłówek oraz danych liczbowych.

Struktura pliku	
rozmiar	element
208	nagłówek
$63 + 4 * liczba_wierszy$	kolumna 1
$63 + 4 * liczba_wierszy$	kolumna 2
...	...
$63 + 4 * liczba_wierszy$	kolumna n

5 Ogólna struktura aplikacji

5.1 Komponenty

Interfejs graficzny aplikacji podzielony jest na "komponenty" - funkcje zawierające logikę kontrolną oraz rysujące pewien element interfejsu. Wywołanie tej funkcji oznacza, że komponent istnieje w obecnej klatce. Funkcje te mogą wywoływać inne funkcje komponentów, co tworzy swego rodzaju strukturę drzewa komponentów, rozpoczynającą się w głównej pętli programu. Pozycja w strukturze drzewa nie ma jednak wpływu na rozmieszczenie elementu, pozycja jest statyczna, lub określana poprzez podane parametry.

5.2 Zarządzanie stanem

Inspirowane działaniem innego drzewiastego systemu komponentów graficznych - webowym Reactem - komponenty, jeśli to konieczne, operują na dwóch rodzajach danych: właściwości (*props*) i stan (*state*).

Właściwości są podawane do funkcji komponentu jako kopia (pass by value), bądź stały wskaźnik, mają za zadanie sterować zachowaniem i wyglądem komponentu.

Stan jest podawany jako wskaźnik do pamięci zarządzanej przez rodzica (komponent wywołujący), ma za zadanie przechowywanie stanu komponentu między wywołaniami, jest modyfikowany głównie przez sam komponent oraz jego dzieci (jeśli zostanie podany do nich jako ich stan).

Stan globalny jest wykorzystywany w niektórych komponentach, które z założenia mają istnieć tylko w jednej sztuce jednocześnie, jak lista otwartych plików lub narzędzia diagnostyczne *FPSCounter* i *Clock*, jako zmienne globalne w jednostce translacyjnej (niewidoczno poza obecnym plikiem .c), lub statyczne zmienne lokalne w funkcji komponentu.

6 Rysowanie wykresu

Raylib zawiera jedynie proste funkcje rysowania kształtów. Jedną z nich jest *DrawSplineLinear*, łącząca podane współrzędne pikseli liniowymi odcinkami. cośtam właśnie jej użyjemy bla

```
DrawSplineLinear(state->point_buffer, state->visible.count, 2.f, DARKBLUE);
```

6.1 Poruszanie się w widoku

Aby zwiększyć interaktywność, użytkownik może się poruszać po płaszczyźnie XY, przeciągając płaszczyzną myszką, oraz przybliżając i oddalając kółkiem myszy. Manipulowane są w ten sposób zmienne skali oraz przesunięcia uwzględnione w przekształceniu punktów ze współrzędnych kartezjańskich na współrzędne ekranu.

6.2 Transformacja współrzędnych

6.2.1 Model matematyczny

Punkt podlega przekształceniu liniowemu:

$$x = \frac{z(x_p - x_v) - x_S}{x_E - x_S} * W + x_O \quad (1)$$

gdzie

z - współczynnik skali

x_p - współrzędna x punktu w układzie kartezjańskim

x_v - przesunięcie x w układzie kartezjańskim

x_S - współrzędna x lewej krawędzi układu kartezjańskiego przy zerowym przesunięciu

x_E - współrzędna x prawej krawędzi układu kartezjańskiego przy zerowym przesunięciu

W - szerokość wykresu w pikselach

x_O - odległość prawej krawędzi wykresu od prawej krawędzi ekranu

$$y = \frac{-z(y_p + y_v) - y_S}{y_E - y_S} * H + y_O \quad (2)$$

gdzie

z - współczynnik skali

y_p - współrzędna y punktu w układzie kartezjańskim

y_v - przesunięcie y w układzie kartezjańskim

y_S - współrzędna y dolnej krawędzi układu kartezjańskiego przy zerowym przesunięciu

y_E - współrzędna y górnej krawędzi układu kartezjańskiego przy zerowym przesunięciu

H - wysokość wykresu w pikselach

y_O - odległość górnej krawędzi wykresu od górnej krawędzi ekranu

Zmiana znaku y_p jest spowodowana, tym że oś $+Y$ we współrzędnych ekranowych skierowana jest w dół.

6.2.2 Implementacja

W kodzie programu zaimplementowane jest to dwuetapowo.

Obliczanie granic widocznego fragmentu

```
Bounds compute_bounds(const float zoom, const Vector2 pan) {
    return (Bounds) {
        .start_x = PLOT_START_X / zoom + pan.x,
        .end_x = PLOT_END_X / zoom + pan.x,
        .start_y = PLOT_START_Y / zoom / ((float) PLOT_WIDTH / PLOT_HEIGHT) + pan.y,
        .end_y = PLOT_END_Y / zoom / ((float) PLOT_WIDTH / PLOT_HEIGHT) + pan.y,
    };
}
```

Transformacja pojedynczego punktu

```
Vector2 transform_v_to_pixel(const Vector2 v, const Bounds bounds) {
    return (Vector2){
        .x = (v.x - bounds.start_x) / (bounds.end_x - bounds.start_x) * PLOT_WIDTH + PLOT_OFFSET_X,
        .y = (-v.y - bounds.start_y) / (bounds.end_y - bounds.start_y) * PLOT_HEIGHT + PLOT_OFFSET_Y,
    };
}
```

Transformacja wielu punktów jednocześnie jest zoptymalizowana pod kątem ilości niepotrzebnych operacji (opisana w punkcie 6.3.3).

6.3 Minimalizacja renderowanych punktów

6.3.1 Opis problemu

Podczas przeglądania wykresu, nie każdy punkt będzie widoczny jednocześnie. Chcąc zminimalizować ilość punktów podlegających transformacji oraz rysowaniu, należy znaleźć zakres punktów widocznych na ekranie.

6.3.2 Szukanie pierwszego punktu

Zakładając, że współrzędna x w zestawie danych jest niemalejąca, możemy potraktować zbiór X jako posortowany zbiór danych i użyć wydajnego algorytmu **wyszukiwania binarnego**, o złożoności obliczeniowej $O(\log_2 n)$.

```

int find_first_visible(const Vector2* data, const int count, const float start_x) {
    if (data[count - 1].x < start_x) return -1;

    int a = 0, b = count - 1;
    while (b - a > 1) {
        const int mid = (a + b) / 2;
        if (data[mid].x > start_x) b = mid;
        else a = mid;
    }
    return a;
}

```

6.3.3 Transformacja punktów

Po znalezieniu pierwszego elementu zbioru wyjściowego, współrzędne są przekształcane i zapisywane w tablicy wyjściowej aż do napotkania punktu znajdującego się za prawą krawędzią widoku lub końca tablicy wejściowej. Wyszukiwanie binarne nie ma tutaj zastosowania, ponieważ i tak każdy widoczny punkt musi zostać przekształcony, co wiąże się z liniowym algorytmem.

```

VisiblePointsInfo compute_visible_points_offset(
    const DataSource data_source, Vector2* point_buffer, const Bounds bounds, const Vector2 offset
) {
    VisiblePointsInfo res = {
        .start = find_first_visible(data_source.data, data_source.count, bounds.start_x - offset.x)
    };
    if (res.start == -1) return res;

    const float x_factor = 1 / (bounds.end_x - bounds.start_x) * PLOT_WIDTH;
    const float y_factor = -1 / (bounds.end_y - bounds.start_y) * PLOT_HEIGHT;

    const int visible_count = data_source.count - res.start;
    for (int i = 0; i < visible_count; i++) {
        const int i_source = i + res.start;
        point_buffer[i].x = (data_source.data[i_source].x - bounds.start_x + offset.x)
            * x_factor + PLOT_OFFSET_X;
        point_buffer[i].y = (data_source.data[i_source].y + bounds.start_y - offset.y)
            * y_factor + PLOT_OFFSET_Y;
        if (data_source.data[i_source].x > bounds.end_x - offset.x) {
            res.count = i + 1;
            return res;
        }
    }
    res.count = visible_count;
    return res;
}

```

Algorytm nie używa funkcji do transformacji pojedynczego punktu, co pozwala na obliczenie niezmiennych współczynników poza pętlą, unikając zbędnych operacji.

6.4 Wykrywanie zmian

Transformacja wszystkich widocznych punktów jest mimo wszystko kosztowną operacją, dlatego wykonywana jest jedynie gdy reprezentacja punktów na wykresie powinna ulec zmianie. Odpowiada za to zmienna *change*, zawierająca flagi sterujące rekalkulacją elementów wykresu, ustawiana przez funkcję odpowiadającą za obsługę wejścia.

7 Analiza danych spęczania

7.1 Charakterystyka danych

Dane zawarte w plikach z pomiaru próby spęczania zawierają 4 kolumny

7.1.1 Load[kN]

Siła zmierzona przez maszynę - opór, który stawia próbka. Wartości ujemne.

7.1.2 Stroke[mm]

Przemieszczenie narzędzia maszyny. Wartości ujemne.

7.1.3 Command[%]

7.1.4 Time[s]

Czas obecnego rekordu, wartość rosnąca od 0, co 0.0025.

7.2 Fazy procesu w danych

- **Brak kontaktu** - obciążenie jest stałe lub z minimalnymi wahaniami, niewielkie
- **Pierwszy kontakt** - obciążenie waha się nierównomiernie, zauważalnie bardziej niż w poprzednim etapie
- **Liniowa** - obciążenie rośnie liniowo
- **Nieliniowa** - wzrost obciążenia stopniowo i płynnie maleje

7.3 Wykrywanie faz

Fazy procesu są na tyle charakterystyczne, że można je wykryć z poziomu algorytmu do napisania jeszcze

7.4 Analiza fazy liniowej

7.5 Analiza fazy nieliniowej

8 Proces optymalizacji algorytmu regresji wielomianowej

8.1 Wstęp

Ten rozdział opisuje proces optymalizacji algorytmu wykonującego regresję wielomianową podanego stopnia dla podanych punktów. Udokumentowane są użyte techniki oraz pomiary czasowe. Przed optymalizacją, algorytm ten był zdecydowanie najbardziej czasochłonną operacją w całym programie.

8.2 Dane wejściowe, sprzęt i środowisko

Podczas testowania przyjęto następujące parametry:

- **Punkty wejściowe:** 29 punktów na krzywej $2\sin x + 2$, dla x od -5, co 0.3
- **Stopień szukanego wielomianu:** 28
- **CPU:** AMD Zen2 Ryzen 3600
- **Kompilator:** GCC 15.2.1
- **Opcje kompilatora w trybie Debug:** -g
- **Opcje kompilatora w trybie Release:** -O3

8.3 Sposób pomiarów

8.3.1 Realny benchmark

Pomiary algorytmu oraz jego fragmentów miały miejsce jako mierzenie części całego działającego programu, maksymalnie raz na klatkę, przy zachowaniu całej jego pozostałej funkcjonalności. Rezultat był wykorzystany do wyświetlenia na żywo wykresu wyznaczonej funkcji, nie ma mowy więc o pomijaniu przez kompilator operacji, których wynik nie jest używany. Dodatkowo w syntetycznym benchmarku, wywołującym w pętli funkcję wiele razy, korzystniej mogłaby być zapełniona pamięć cache oraz bardziej dokładny predyktor skoków, dlatego wplecenie pomiarów w standardowy przepływ programu lepiej oddaje wydajność w realnych warunkach.

8.3.2 RDTSC

Architektura x86 implementuje instrukcję **RDTSC** (*Read Time-Stamp Counter*), zwracającą obecną wartość licznika cykli. Licznik służy do precyzyjnego pomiaru czasu i jest inkrementowany co stałą jednostkę czasu, odpowiadającą częstotliwości procesora. Dynamiczne taktowanie obecne we współczesnych rdzeniach nie ma wpływu na częstotliwość tego licznika. W przypadku sprzętu pomiarowego jest to 3.6GHz.

8.3.3 Implementacja pomiaru

W momencie uruchomienia benchmarku ustawiana jest flaga (pkt. 6.4), zapewniająca że komponent wielomianu otrzyma zmianę w każdej klatce, co wywoła pożądaną funkcję. Flaga będzie ustawiana, dopóki benchmark nie zbierze 10000 pomiarów.

Rozpoczęcie pomiaru

```
void clock_start() {  
    #ifndef __arm64  
        start_cycles = __rdtsc();  
    #endif  
    start = clock();  
}
```

Funkcja ustawia zmienne globalne licznika cykli używając funkcji wewnętrznej `__rdtsc` oraz drugiego licznika używając kanonicznego `clock`. Użycie `__rdtsc` jest wyłączone na platformie ARM, gdzie instrukcja RDTSC nie jest dostępna.

Zakończenie pomiaru

```
void clock_end() {
#ifdef __arm64
    end_cycles = __rdtsc();
#endif
    end = clock();
    const clock_t diff = end - start;
    const unsigned long long diff_cycles = end_cycles - start_cycles;

    if (samples == -1) return;
    if (samples < CLOCK_SAMPLE_COUNT) {
        samples++;
        sum += diff;
        sum_cycles += diff_cycles;
        if (best == -1 || diff < best) best = diff;
        if (best_cycles == -1 || diff_cycles < best_cycles) best_cycles = diff_cycles;

        if (samples == CLOCK_SAMPLE_COUNT) {
            /* wyświetlanie wyniku... */
            samples = -1;
            sum = 0;
            sum_cycles = 0;
            best = -1;
            best_cycles = -1;
        }
    }
}
```

Liczona jest różnica w z obecnym a poprzednim stanem licznika. Wartość -1 dla `samples` oznacza wyłączony benchmark, a dla `best` i `best_cycles` - brak wartości.

8.3.4 Narzut pomiaru

Aby móc wyciągnąć wnioski z pomiarów sekcji kodu, należy najpierw zmierzyć czas wykonania samego kodu pomiarowego. Wszelkie wyniki zbliżone do narzutu powiaru są niemiernodajne.

Debug					
Pomiar	Liczba próbek	Średnia RDTSC	Min RDTSC	Średnia clock [ms]	Min clock [ms]
1	10000	3642	2592	0.000814	0.000000
2	10000	3561	2484	0.000791	0.000000
3	10000	3604	2520	0.000814	0.000000
5	10000	3683	2592	0.000810	0.000000

Release					
Pomiar	Liczba próbek	Średnia RDTSC	Min RDTSC	Średnia clock [ms]	Min clock [ms]
1	10000	3880	2628	0.000817	0.000000
2	10000	3615	2556	0.000789	0.000000
3	10000	3713	2736	0.000796	0.000000
4	10000	3569	2700	0.000775	0.000000

Czas jest na tyle mały, że `clock` wskazuje 0 - jest za mało precyzyjny. Co ciekawe, wygląda na to, sam narzut w konfiguracji release jest nieco wolniejszy niż w debug.

8.4 Pierwotny algorytm

8.4.1 Kod

```
void regression(CurvePolynomial* curve, const Vector2* points, const int point_count) {
    clock_start();
    // przygotowanie
    const int degree = point_count - 1;
    const int n = point_count;
    const int m = degree;

    float* matrix = malloc((m*m + 2*m) * sizeof (float) + m * sizeof (int));
    float* errors = &matrix[m*m];
    float* coeffs = &errors[m];
    int* swap_table = (int*) &coeffs[m];

    // wypełnianie macierzy i wektora
    for (int row = 0; row < m; ++row) {
        swap_table[row] = row;

        for (int col = 0; col < m; ++col) {
            float cell = 0;
            const int power = row + col;
            for (int i = 0; i < n; ++i) {
                cell += pownf(points[i].x, power);
            }
            _get_mat_cell(row, col) = cell;
        }
        float error = 0;
        for (int i = 0; i < n; ++i) {
            error += pownf(points[i].x, row) * points[i].y;
        }
        _get_error(row) = error;
    }

    // eliminacja Gaussa - postępowanie proste
    for (int i = 0; i < m - 1; ++i) {
        float max = fabsf(_get_mat_cell(i, i));
        int max_row = i;
        for (int row = i + 1; row < m; ++row) {
            const float current_abs = fabsf(_get_mat_cell(row, i));
            if (current_abs > max) {
                max = current_abs;
                max_row = row;
            }
        }
        if (max_row != i) swap_rows(swap_table, i, max_row);

        const float cell = _get_mat_cell(i, i);
        for (int row = i + 1; row < m; ++row) {
            const float coeff = _get_mat_cell(row, i) / cell;
            _get_error(row) -= coeff * _get_error(i);
            for (int col = 0; col < m; ++col) {
                _get_mat_cell(row, col) -= coeff * _get_mat_cell(i, col);
            }
        }
    }
}
```

```

// eliminacja Gaussa - postępowanie odwrotne
_get_coeff(m - 1) = _get_error(m - 1) / _get_mat_cell(m - 1, m - 1);
for (int i = m - 2; i >= 0; --i) {
    float sigma = 0;
    for (int k = i + 1; k < m; ++k) {
        sigma += _get_mat_cell(i, k) * _get_coeff(k);
    }
    _get_coeff(i) = (_get_error(i) - sigma) / _get_mat_cell(i, i);
}

// przepisanie wyników i zwolnienie pamięci
for (int i = 0; i < m; ++i) {
    curve->coefficients[i] = _get_coeff(i);
}
free(matrix);

curve->order = m;
curve->start_x = points[0].x;
curve->end_x = points[point_count - 1].x;
clock_end();
}

```

8.4.2 Benchmark

Debug					
Pomiar	Liczba próbek	Średnia RDTSC	Min RDTSC	Średnia clock [ms]	Min clock [ms]
1	10000	1228592	1182240	0.340353	0.328000
2	10000	1222491	1187676	0.338560	0.329000
3	10000	1221949	1184508	0.338329	0.328000
4	10000	1219704	1187136	0.337932	0.328000

Release					
Pomiar	Liczba próbek	Średnia RDTSC	Min RDTSC	Średnia clock [ms]	Min clock [ms]
1	10000	951561	913932	0.263545	0.253000
2	10000	952509	913644	0.263831	0.253000
3	10000	939618	913608	0.260251	0.252000
4	10000	942651	914544	0.261089	0.253000

8.4.3 Wnioski

Widać zauważalną różnicę między konfiguracją debug, a release. Wykonanie mieści się poniżej 1 mili-sekundy, jednak zajmuje dużą ilość czasu w porównaniu do reszty programu oraz powoduje zauważalne spadki liczby klatek na sekundę (z ok. 8000 do ok. 1000).